Gestão de Projecto de Software

Licenciatura em Engenharia Informática

2024/2025

Traditional Process Models

Waterfall Methodology

A10

310

Modelos de desenvolvimento de Software – 2 caminhos …

Traditional Process Models

- Waterfall Methodology



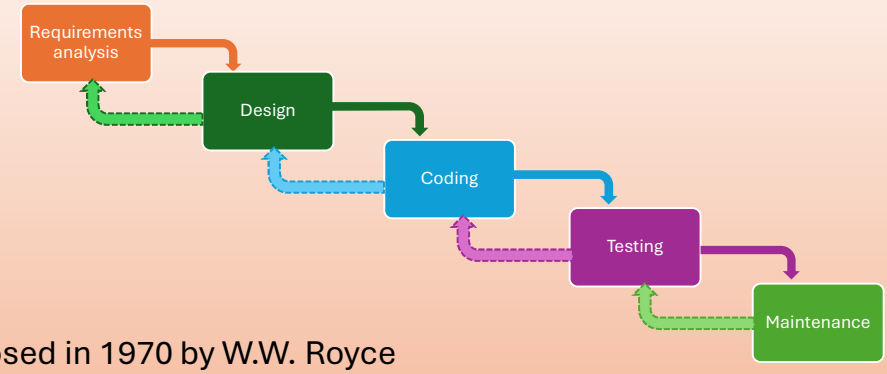
Uma cascata ….

Agile Software Development



311

Waterfall lifecycle



- Proposed in 1970 by W.W. Royce
- Models a systematic sequential approach
  - But real projects rarely follow the sequential flow
  - When something goes wrong, go back to a previous step (expensive)

312

Analysis

- Requirements must be specified in the first step
  - Difficult to accomodate the natural uncertainty at the beginning of the projects
- Models a systematic sequential approach
  - Real projects rarely follow the sequential flow
- The result of each phase is one or more approved documents
  - Document-driven approach – costly iterations
  - Prematurely freezing requirements may mean the system doesn't do what the user wants
- Works well when requirements are well understood, and technologies known

313

Vantagem da metodologia Waterfall

- Estrutura e Clareza
  - A sequência bem definida de fases facilita o planeamento e a organização do projeto.
- Documentação Completa
  - Cada fase gera a documentação detalhada, útil para referência futura e para a integração de novos membros na equipa.
- Gestão Previsível
  - A abordagem sequencial permite uma gestão de projecto mais previsível e organizada.

Desvantagens da metodologia Waterfall

- Baixa Flexibilidade
  - É difícil introduzir mudanças depois da fase de requisitos.
- Entrega Demorada
  - Como o produto final só é entregue no fim do processo, os utilizadores têm de esperar até a conclusão de todas as fases para verem o resultado.
- Risco de Erros por Requisitos Incorrectos
  - Caso haja erros ou lacunas na fase de requisitos, podem afectar negativamente o projecto inteiro, pois todas as fases seguintes dependem desta definição inicial.

O modelo Waterfall é adequado para projectos onde os requisitos são bem definidos desde o início e pouco sujeitos a alterações

1 – Análise de requisitos

- Todos os requisitos necessários para o projeto são recolhidos e documentados
  - inclui entrevistas com as partes interessadas
  - estudos de mercado e o levantamento detalhado dos requisitos funcionais e técnicos
- O objectivo é ter uma visão clara de todas as funcionalidades que o sistema deverá ter, de forma a garantir que os requisitos são compreendidos por toda a equipa
- O produto final desta fase é um documento de especificação de requisitos, que serve como base para as fases seguintes

Software Requirements

- *Software requirements* are documentation that completely describes the behavior that is required of the software, before the software is designed, built and tested.
  - Requirements analysts (or business analysts) build software requirements specifications through *requirements elicitation*. Ex:
    - Interviews with the users, stakeholders and anyone else whose perspective needs need to be considered during the design, development and testing of the software
    - Observation of the users at work

Use Cases

- A *use case* is a description of a specific interaction that a user may have with the system.
- Use cases are simple tools for describing the functionality of the software.
  - Use cases do not describe any internal workings of the software, nor do they explain how that software will be implemented.
  - They simply show the steps that the user follows to use the software to do his work.
  - They also describe all the ways that the users interact with the application.

Functional Requirements

- *Functional requirements* define the outward behavior required of the software project.
  - The goal of the requirement is to communicate the needed behavior in a clear and unambiguous manner.
  - The behavior in the requirement can contain lists, bullets, equations, pictures, references to external documents, and any other material that will help the reader understand what needs to be implemented.

Nonfunctional Requirements

- *Nonfunctional Requirements* define characteristics of the software which do not change its behavior.
  - Users have implicit expectations about how well the software will work.
  - These characteristics include how easy the software is to use, how quickly it executes, how reliable it is, and how well it behaves when unexpected conditions arise.
  - The nonfunctional requirements define these aspects about the system.
    - The nonfunctional requirements are sometimes referred to as “non-behavioral requirements” or “software quality attributes”

UI Mockups

- **UI mockups** are used to show the stakeholders or the developers how the software will look like
- Mockups can range from very simple hand drawn screen layouts, through realistic bitmaps, to semi-functional user interfaces developed in a software development tool
  - Low-fidelity prototypes are a quick and easy way to translate high-level design concepts into tangible and testable artifacts
  - High-fidelity prototypes appear and function as similar as possible to the actual product
- **Mockups are an easy and powerful way to transmit UI requirements**

Software Requirements Specification

- The **Software Requirements Specification (SRS)** represents a complete description of the behavior of the software to be developed.
- The SRS includes:
  - A set of **use cases** that describe all the interactions that the users will have with the software.
  - All the **functional requirements** necessary to define the internal workings of the software: calculations, technical details, data manipulation and processing, and other specific functionality that shows how the use cases are to be satisfied
  - **Nonfunctional requirements**, which impose constraints on the design or implementation (such as performance requirements, quality standards or design constraints).
  - **Mockups** to show the UI elements of the application

322

Scope *vs* Requirements *vs* Design

- **Scope** demonstrates the needs of the organization, and is documented in a vision and scope document
- **Requirements** document the behavior of the software that will satisfy those needs
- **Design** shows how those requirements will be implemented technically

323

2 - Projeto (Design)

- O objectivo geral é criar uma base sólida que guie o desenvolvimento sem a necessidade de grandes alterações mais tarde
- A fase de design é onde se planeia toda a arquitectura do sistema
  - **2.1 - Design de alto nível**
    - estrutura geral e módulos principais
  - **2.2 - Design detalhado**
    - Especificações dos componente individuais
- Design da interface gráfica homem/máquina
- Design da estrutura de base de dados (caso a aplicação obrigue)
- Escolha das tecnologias
- Escolha das ferramentas a usar

2.1 - Design de alto nível (arquitectura do sistema)

- Definição da arquitectura geral do sistema
  - Estrutura e organização dos principais componentes e módulos
    - Definição **como o sistema será estruturado e como os diferentes componentes vão interagir entre si**
- São tomadas decisões sobre a divisão em módulos ou subsistemas e a escolha dos padrões arquitecturais
  - Arquitectura em camadas, cliente-servidor, ...
- São definidas as questões relacionadas com a **integração** entre os diferentes componentes e a **conectividade** entre os módulos (incluindo os dados que circulam dentro do sistema e como o sistema se vai integrar com sistemas externos)
- São definidas as **tecnologias a serem usadas**
  - **Linguagem de programação, base de dados, framworks, bibliotecas ....**
  - A escolha tem em conta factores como o desempenho, segurança, familiaridade da equipa com as ferramentas ...
- O produto final desta subfase é um **documento de arquitectura** ou **documento de design de alto nível**
  - Onde é apresentado a visão geral do sistema e orienta o trabalho da subfase seguinte, onde os detalhes serão aprofundados

2.2 - Design de Baixo Nível (Design Detalhado)

- Cada componente do sistema é especificado minuciosamente
- Enquanto o design de alto nível fornece uma visão geral da arquitectura do sistema, **o design de baixo nível foca-se nos detalhes de implementação de cada módulo e componente**
- Definição da estrutura de dados interna dos módulos, dos algoritmos específicos que serão utilizados, a lógica de cada função e o fluxo de execução dentro de cada componente
- Definição dos protocolos de comunicação entre os componentes e os formatos de dados a serem transmitidos, garantindo que todos os módulos possam interagir sem erros ou inconsistências
- O design da **interface gráfica**
- **modelo de dados** que deve incluir a estrutura das tabelas, os campos e as relações entre entidades
  - Esta parte é essencial para garantir a consistência e a integridade dos dados no sistema
- O produto final desta subfase é um conjunto de **documentos de design detalhado** que descrevem cada componente do sistema de forma pormenorizada
  - Estes documentos têm que conter todas as especificações necessárias para ser possível transformar o design em código

326

326

3 – Codificação / Implementação / Desenvolvimento

- O software é codificado de acordo com o design estabelecido
- São desenvolvidas as componentes e funcionalidades descritas nos documentos de design e requisitos
- Na metodologia Waterfall, esta fase é realizada de uma forma rigorosa e sem improvisos, uma vez que todos os detalhes foram previamente planeados
- O produto final é o código-fonte completo do sistema
- O progresso é monitorizado para garantir que se respeitam os requisitos e especificações iniciais

327

327

3 – Codificação / Implementação / Desenvolvimento – Algumas fases

3.1 - Preparação e Configuração do Ambiente de Desenvolvimento

- A equipa de desenvolvimento prepara o ambiente de desenvolvimento
  - configuração das ferramentas de desenvolvimento
    - IDE (Ambiente de Desenvolvimento Integrado)
    - sistemas de controlo de versão (por exemplo, Git)
    - bibliotecas e dependências necessárias
    - criação de uma estrutura de pastas e arquivos organizados
  - Configurar um ambiente robusto permite que a equipa trabalhe de forma colaborativa e organizada, facilitando o controlo de versões e a integração de novos elementos na equipa (caso seja necessário)

328

328

3 – Codificação / Implementação / Desenvolvimento – Algumas fases

3.2 - Divisão do Trabalho por Módulos e Tarefas

- Com base na documentação da fase de design o sistema a desenvolver é dividido em módulos e tarefas atribuídas a diferentes programadores ou equipas especializadas.
  - Permite que o trabalho seja executado em paralelo, aumentando a velocidade de desenvolvimento
- Cada módulo deve ser codificado segundo o design detalhado, garantindo que cada componente segue as especificações e se integra corretamente com os restantes módulos do sistema.

329

329

3 – Codificação / Implementação / Desenvolvimento – Algumas fases

3.3 - Codificação de Funcionalidades e Módulos

- Os programadores escrevem o código para cada funcionalidade e módulo conforme as especificações do design detalhado.
- É fundamental seguir boas práticas de programação
  - Escrever código legível e documentado
    - Usar comentários e uma estrutura de código clara para facilitar a compreensão e a manutenção futura
  - Utilizar nomes descritivos para variáveis, funções e classes, de modo a tornar o código autoexplicativo
  - Modularidade
    - Implementar o código de forma modular, garantindo que cada componente é autónomo e executa uma função específica
  - Reutilização de Código
    - Sempre que possível, os programadores reutilizam código já escrito para evitar duplicação e reduzir erros.

330

330

3 – Codificação / Implementação / Desenvolvimento – Algumas fases

3.4 - Implementação de Estruturas de Dados

- Desenvolvimento e implementação da estrutura de dados necessária (caso se aplique)

331

331

3 – Codificação / Implementação / Desenvolvimento – Algumas fases

3.5 - Integração de Componentes e Testes Unitários

- À medida que os módulos individuais são desenvolvidos, são realizados **testes unitários**
- Após a codificação dos módulos individuais, a equipa de desenvolvimento realiza a **integração dos componentes**, unindo os módulos de forma que o sistema funcione como um todo
  - Nesta etapa, podem surgir problemas de integração, que precisam de ser identificados e resolvidos

3 – Codificação / Implementação / Desenvolvimento – Algumas fases

3.6 - Gestão e Controlo de Versões

- Durante a fase de codificação, é essencial utilizar um sistema de controlo de versões, como o Git, para gerir o código-fonte. Estes sistemas permitem que a equipa acompanhe as alterações no código, identifique e reverta problemas, e garanta que todos os programadores estão a trabalhar na versão mais recente do projeto

4 – Testes / Verificação

- São executados:
  - testes unitários
  - testes de integração
  - testes de aceitação
  - testes de usabilidade
- O objectivo é identificar e corrigir os erros antes de ser disponibilizado o software
- Garantir que todas as funcionalidades funcionam correctamente
- O produto final desta fase é um sistema testado e pronto a ser implementado

334

334

5 – Manutenção

- Esta fase assegura que o software se mantem operacional e relevante ao longo do tempo
- A manutenção inclui:
  - Correcções de defeitos
  - Melhorias de desempenho
  - Adaptações a novas necessidades tecnológicas ou de negócio
- A manutenção é um processo contínuo

335

335

Tipos de manutenção (1) - Manutenção Correctiva

- **Manutenção Correctiva**
  - identificar e corrigir erros que foram encontrados pelos utilizadores após a implementação do sistema
  - Por vezes, alguns problemas escapam aos testes iniciais, sendo necessário agir rapidamente para resolver essas falhas sem comprometer a operação do sistema.

336

336

Tipos de manutenção (2) - Manutenção Adaptativa

- **Manutenção Adaptativa**
  - Alteração do ambiente onde o software é executado muda (por exemplo, sistema operativo, hardware, ou plataformas de terceiros), o software precisa de ser actualizado para se adaptar a essas alterações
  - A manutenção adaptativa foca-se em **ajustar o sistema para que continue a funcionar em novos ambientes** ou com novas versões de software e hardware

337

337

Tipos de manutenção (3) - Manutenção de optimização

- **Manutenção de optimização**
  - Modificações são feitas para melhorar o desempenho, a usabilidade ou a eficiência do sistema, mesmo que esteja a funcionar correctamente
  - Inclui a adição de pequenas funcionalidades ou a melhoria da experiência do utilizador, sem alterar o funcionamento essencial do sistema

338

338

Tipos de manutenção (4) - Manutenção preventiva

- visa **antecipar e prevenir possíveis problemas futuros**
- Envolve a análise do sistema e a realização de alterações que minimizam a probabilidade de ocorrerem problemas, prolongando a vida útil e a fiabilidade do sistema.
- É uma forma de garantir que o software continua robusto e capaz de lidar com potenciais riscos

339

339

Processo de Manutenção Correctiva (1)

1 - Identificação e Análise de Problemas

- Os utilizadores comunicam à equipa de desenvolvimento os problemas ou novas necessidades, geralmente através de um sistema de tickets ou suporte técnico
- A equipa analisa os problemas ou solicitações para determinar se são viáveis e se estão dentro do âmbito do do sistema
- Caso seja identificado um problema, este é priorizado com base na sua gravidade e impacto.

340

340

Processo de Manutenção Correctiva (2)

2 – Planeamento das modificações

- Elaboração de um plano detalhado, onde é descrito as alterações e os recursos necessários e uma previsão de tempo de implementação
- O planeamento é essencial, pois cada alteração deve ser cuidadosamente avaliada para evitar impactos nas funcionalidades já existentes.
  - Cada alteração passa por uma análise de impacto de forma a prever possíveis “efeitos colaterais”

341

341

Processo de Manutenção Correctiva (3)

- **Implementação da alterações**
  - São realizadas as alterações no código conforme as especificações detalhadas

342

342

Processo de Manutenção Correctiva (4)

- **Testes de verificação e validação**
  - Após a implementação são efectuados os devidos testes para garantir:
    - Que o problema foi resolvido
    - Que o sistema continua a funcionar correctamente

343

343

Processo de Manutenção Correctiva (5)

- **Documentação das alterações**
  - Cada modificação é documentada para manter um registo histórico
  - A documentação inclui detalhes sobre o problema resolvido, o código alterado, os testes realizados e os seus resultados

344

344

Comparação Resumida entre Waterfall e Scrum

Característica	Waterfall	Scrum
Tipo de Projecto	Estável, com requisitos claros e fixos	Dinâmico, com requisitos mutáveis
Documentação	Extensa e detalhada	Leve, foco na comunicação directa
Gestão de Mudanças	Difícil e dispendiosa	Adaptável, flexível e ágil
Tempo de Entrega	Mais longo, entrega no final	Entregas incrementais em cada sprint
Requisitos do Cliente	Claros desde o início	Evoluem e ajustam-se durante o projecto
Feedback do Cliente	No final do desenvolvimento	Contínuo, a cada sprint
Ambiente de Desenvolvimento	Formal e estruturado	Colaborativo e iterativo
Exemplos de Aplicação	Sistemas críticos e regulamentados ...	Aplicações B2C, produtos de inovação rápida ....

345

345


isec
Politécnico de Coimbra
Gestão de Projecto de Software
Licenciatura em Engenharia Informática
2024/2025
A11
346

346